2022 IEEE International Conference on Robotics and Automation (ICRA)
May 23-27, 2022. Philadelphia, PA, USA

Multi-UAV Disaster Environment Coverage Planning with Limited-Endurance

Hongyu Song*†, Jincheng Yu*§, Jiantao Qiu§, Zhixiao Sun¶, Kuijun Lang¶, Qing Luo¶, Yuan Shen§ and Yu Wang§

***Abstract*— Disaster areas involving floods and earthquakes are commonly large, with the rescue time being quite tight, suggesting multi-Unmanned Aerial Vehicles (UAV) exploration rather than employing a single UAV. For such scenarios, current UAV exploration is modeled as a Coverage Path Planning (CPP) problem to achieve full area coverage in the presence of obstacles. However, the UAV's endurance capability is limited, and the rescue time is constrained, prohibiting even multiple UAVs from completing disaster area coverage on time. Therefore, this paper defines a multi-Agent Endurance-limited CPP (MAEl-CPP) problem that is based on an *a priori* known heatmap of the disaster area, which affords to explore the most valuable areas under UAV limited energy constraints. Furthermore, we propose a path planning algorithm for the MAEl-CPP problem by ranking the possible disaster areas according to their importance through satellite or remote sensing aerial images and completing path planning according to this ranking. Experimental results demonstrate that the search efficiency of the proposed algorithm is 4.2 times that of the existing algorithm.**

I. Introduction

Multi-UAV exploration is more efficient than single UAV [1] in disaster relief work where the area, such as in floods and earthquakes, to be explored is usually large and the rescue is time-limited [2]. Using the rescued survivor number and the time used as the evaluation metric, Yang et al. [3] proves that the average time to rescue each survivor significantly reduces as the number of UAVs increases.

Existing UAV exploration methods model the multi-UAV exploration task as a coverage path planning (CPP) problem [4], where the UAVs' starting point is arbitrary within a region containing no-fly zones and obstacles. However, as the UAVs cruise at least 30-40 meters from the ground [5], a real-world rescue scenario almost without obstacles at that altitude force this type of CPP modeling to be inapplicable.

Additionally, a UAV's endurance is limited [6], while the rescue time is quite constrained. Thus, despite utilizing multiple UAVs, complete on-time disaster area coverage is not guaranteed. For example, Zhengzhou flooded this year, with its Zhongyuan District being 193 square kilometers, requiring 20 UAVs operating for about 6.9 hours to achieve uninterrupted coverage. This is far more than the 1-hour no-load endurance of standard multi-rotor UAVs. At the same time, the 6.9 hours task time exceeds the safe survivor's survival time of specific disasters. Therefore, considering a UAV's limited endurance, it must explore places first where dangers are more likely to occur, e.g., the probability of a fire in a stadium is significantly greater than that of a lake. Therefore, we define a multi-Agent Endurance-limited CPP (MAEl-CPP) problem based on an *a priori* known heatmap [7] of the disaster area that affords to explore the most valuable areas (goals) under a limited energy strategy. Hence, trapped people or disaster-stricken property have a greater chance of being rescued and protected. Currently, pre-disaster maps/satellite images are exploited to detect and divide valuable areas. Nevertheless, calculating the optimal solution covering as many important goals as possible within a specific range driven by the UAV's endurance constrain is quite complicated [8]. Using multiple robots to cover the target area is even more challenging. Simple path coverage algorithms, e.g., Zig-Zag continuous search [9] do not distinguish targets, and multiple robots are prone to path repetition. Accordingly, the naive greedy algorithm will fall into a local optimum solution, and the robots will interfere with each other during the multi-robot collaboration prohibiting this method from solving the MAEl-CPP problem.

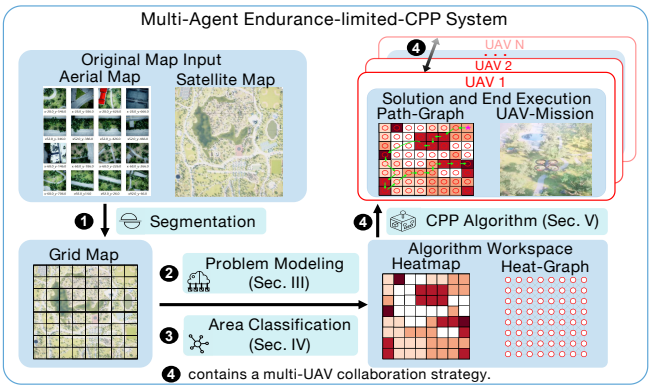


Fig. 1: Overview of our MAEl-CPP framework.

To address the above problems, we propose a multi-robot exploration framework illustrated in Fig. 1. The contributions of our work can be summarized to:

- Formulating a multi-Agent Endurance-limited CPP (MAEl-CPP) problem suitable for various potentially

* These authors contribute equally.

† Department of Electrical and Computer Engineering, Technische Universität München, Munich, Germany. This work is during his intership at Tsinghua University.

§ EE department, Tsinghua University. Beijing, China. {yu-wang, yu-jc}@tsinghua.edu.cn

¶ Avic Shenyang Aircraft Design And Research Institute. Shenyang, China.

This research was supported by National Natural Science Foundation of China (No. U19B2019, M-0248), Tsinghua-Meituan Joint Institute for Digital Life, Tsinghua EE Independent Research Project, Beijing National Research Center for Information Science and Technology (BNRist), and Beijing Innovation Center for Future Chips.

2022 IEEE International Conference on Robotics and Automation (ICRA) | 978-1-7281-9681-7/22/$31.00 ©2022 IEEE | DOI: 10.1109/ICRA46639.2022.9812201

Authorized licensed use limited to: Hanoi University of Science & Technology. Downloaded on October 23,2026 at 08:26:45 UTC from IEEE Xplore. Restrictions apply.

disaster-affected environments.

- A method generating *a priori* heatmap of the pre-disaster environment exploiting a single environmental satellite map or a remote aerial map.
- We propose several path planning algorithms that are appropriate for MAEl-CPP. Among them, the SVReC presents an outstanding performance in search efficiency, reaching 4.2 times of Zig-Zag search.

The remainder of this paper is as follows. Section III-A presents our defined problem. Then, Section IV presents the method to convert the original map into a heatmap, while Section V generates the optimal path according to the *a priori* heatmap. Then, based on the above problems and algorithms Section VI introduces the experimental setup. Finally, Section VII concludes this work and suggests future work.

II. Related Work

Coverage path planning (CPP) has been widely discussed in the robotics community, the Backtracking Spiral Algorithm (BSA) [10] is a classic work. This algorithm can completely cover an environment considering the occupied cells. Accordingly, Gabriely et al. [11] propose a non-repetitive closed-loop coverage involving a closed environment containing obstacles based on competitive tours [12]. M-STC* [13] expands the work of Gabriely et al. and suggests a solution that reasonably allocates the load distribution [14] of multiple robots. Torres et al. [15] develop a Zig-Zag-like path to complete the path coverage by targeting 3D reconstruction, which can work in convex and non-convex environments. Given that modeling these traditional CPP problems consider covering the entire map, these are not suitable for the MAEl-CPP problem. This is because MAEl-CPP aims to maximize the search efficiency rather than cover the entire map, as physical constraints make it challenging to cover it.

Commonly, the map employed for the CPP problem follows a particular distribution, e.g., BSA deals with path planning by dividing the grid cells into two types, covering and non-covering, or Panagou et al. [16] distinguish viewpoints into interested and uninterested. An et al. [17] introduce the prior distribution map to cover the area of interest. Accordingly, Lin et al. [18] utilize a probability map based on Gaussian distributions and suggest an algorithm solving the single-robot CPP problem for such a type of map. However, when the disaster occurs, the above maps cannot describe a particular area's specific *a priori* conditions. The reason being the range of the real-world disaster hazard does not deliberately follow a specific distribution, and for the current types of maps, it is not trivial to deploy specific search efficiency-oriented tasks. Therefore, a map constructed according to the potential disaster risks of various real environments is mandatory for the MAEl-CPP problem.

Furthermore, some CPP algorithms adopt physical constraints. For example, Yang et al. limit the time searching for survivors, while Lin et al. use the time step of CPP to limit energy consumption, with each waypoint adding a time step. Similarly, m-STC* introduces c as the robot's capacity, which is the sum of the total nodes in a spanning tree, with Franco et al. [19] considering a robot's more peculiar features. In any case, the UAV's energy consumption limit must be rigorously stated, but each UAV type has its efficiency and performance governing parameters. Hence, the energy consumption of the MAEl-CPP must be easily extended to different UAVs. Therefore, we define the endurance as the total path length calculated through the Euclidean distance between waypoints.

III. Problem Definition & Preliminaries

A. MAEl-CPP Problem

This section defines a new MAEl-CPP problem that sets the potential disaster level by utilizing representative features from the entire map. The disaster level should describe the probability of a specific disaster to occur in an area and conform to the common sense of search and rescue problems. If these requirements are fulfilled, we can deploy UAVs in a more realistic simulation or real environment. Then, according to the map's representative features, the grid map of the task space will be converted into a heatmap. The latter is a concise and informative map showing the differences between cells. Further details on deriving a heatmap from a grid map are introduced in Section IV.

In the problem examined here, we employ a graph structure [20] called Heat-Graph $\mathcal{H}$, with its vertices corresponding to the heatmap cells on a one-to-one basis that share the same positional relationship (Fig. 2). For example, given cell C_{xy} and vertex η_{ij}, whose vertex weight is called Efficiency weight r, xy denote the coordinates of cell center and ij is the row and column in the graph structure, $r(\eta_{ij})$ is mapped from the heat-value $h(C_{xy})$ of the heatmap illustrated in Fig. 2 (b). As a physical limitation, we stipulate that the endurance (Maximum path length) of each UAV is $\mathcal{D}$. The edge weight $e(\eta_{ij}, \eta_{kl})$ between neighbor vertices is presented in Fig. 2 (a). For a heatmap with grid size of 40mx40m, we obtain:

$$\eta_{ij} = \mathcal{G}(C_{xy}), i = (y_{max} - y)/40 + 1, j = x/40 + 1 \quad (1)$$

$$e(\eta_{ij}, \eta_{kl}) = \sqrt{(i-k)^2 + (j-l)^2} \quad (2)$$

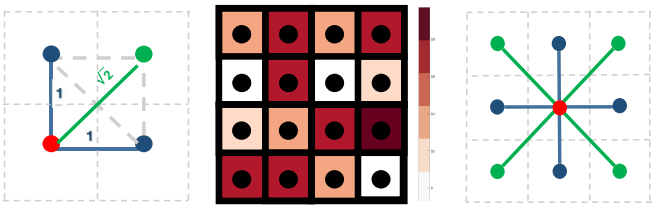
$$r(\eta_{ij}) = \begin{cases} 10 & h(C_{xy}) = 1.0 \\ 10h(C_{xy}) - 2 & else \end{cases} \quad (3)$$

where $\mathcal{G}$ is the map created after converting the heatmap to the Heat-Graph, and the max accumulated edge weight is $d_m = D/40$. We map $h(C_{xy})$ to $r(\eta_{ij})$ utilizing Equ. (3) to enhance the importance of one class and impose one of the remaining classes not to be covered.

Given $\mathcal{H}$ and n UAVs, the flight trajectory of the ith UAV U_i is $\Gamma_i = \mathcal{G}(\gamma_i)$, where γ_i is a directed sub-graph of $\mathcal{H}$ and contains M ηs. R_i and d_i are the accumulated Efficiency weight (Eff-weight) and edge weight, respectively. R can be used to describe the overall search efficiency of a path. Then solving the path-graph γ_i process can be summarized as:

$$\arg\max_{\gamma_i} R_i, \quad \text{s.t.} i \leq n, d_i \leq d_m \quad (4)$$

Authorized licensed use limited to: Hanoi University of Science & Technology. Downloaded on October 23,2026 at 08:26:45 UTC from IEEE Xplore. Restrictions apply.



(a) Edge Weight (b) Efficiency Weight (c) Neighbor Verteces

Fig. 2: Edge Weight and Efficiency Weight. (a) Edge weight e . The e of the vertices adjacent to the side of the cell is 1, and the e corresponding to the vertex whose cell position is diagonal is $\sqrt{2}$. (b) The relationship between the Efficiency weight r and the heat-value (color) of the cell. The higher the heat-value in the heatmap, the higher the Efficiency weight r in the Heat-Graph. Further details on the heatmap are presented in Sec. IV. (c) The eight neighbor vertices of a vertex, where red is the current vertex, green and blue are the neighbor vertices of the diagonal and translation, respectively.

$$R_i = \sum_{m=1}^{M} r(\eta_m), d_i = \sum_{m=1}^{M-1} e(\eta_m, \eta_{m+1}), \quad \eta_m \in \gamma_i \quad (5)$$

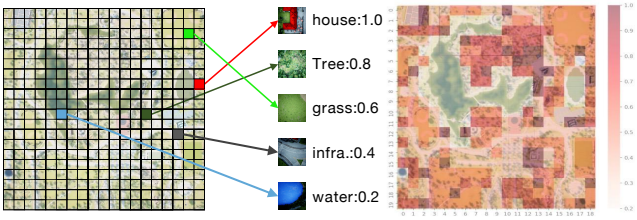
B. Naive Greedy Path Coverage For a Single UAV

After briefly describing the path coverage problem, we first introduce a relatively simple solution inspired by the greedy algorithm [21], which is a classic method to solve NP-Hard problems [22]. Specifically, we suggest the Naive Greedy (Na-Greedy) Path Coverage algorithm involving the following calculation steps. Given a Heat-Graph $\mathcal{H}$, a UAV start vertex η_s and the max accumulated edge weight d_m, we obtain after path planning γ_i, R, and the cover rate σ, which is the percentage of covered vertices out of all vertices. The pseudo-code of the Na-Greedy algorithm is presented in Fig. 4 Algorithm 1.

Although the proposed Na-Greedy algorithm can quickly solve some MAEl-CPP problems from a Heat-Graph, in some cases, due to the hard constraints of the CLOSE set [23], which is the set of covered vertices. The UAV path may fall into a local optimal solution, i.e., all vertices around the UAV are in the CLOSE, and the UAV can not continue to plan the path after exiting the loop. To solve this problem, we propose the Heu-Greedy and SVReC algorithms introduced in Section V.

IV. Potential Disaster Heatmap

We efficiently generate the proposed map and assign different Eff-weights (Section III-A) to the cells containing different scenes, e.g., water and house scenes. Utilizing the proposed map for the actual MAEl-CPP problem requires building the map such as each cell is based on the probability of the occurrence of the actual disaster. However, it is challenging to create such a map, as most current maps considering the CPP problem focus on the grid occupancy [24] or the probability distribution as described in Section II. Thus, an image map from the real environment cannot be converted into the desired map type, and the existing map



(a) Grid Map & Class (b) Heatmap

Fig. 3: Grid-cell Classification & Heatmap. Classification is described next to the sample picture in the form of **class: heat-value**.

types are inefficient in searching and rescuing the post-disaster environment. The latter is due to the lack of current map types' of comprehending the potential disaster probability of various scenarios and the corresponding danger level after the disaster.

Additionally, the map must also guide the UAV's low-altitude flight during the search and rescue process. Therefore, we segment into a grid map a high-altitude satellite image fig. 3 (a) of our simulation scene. The grid map cells are classified into five classes according to the possibility that people in the scene are trapped or in danger when the fire occurs. It should be noted that classification is based on common sense, while several other criteria can be utilized depending on the scenario or the type of disaster, affording a universal classification concept.

Once the heatmap described above is generated, the first challenge is classifying the cells. Large-area city maps often contain tens of thousands of cells that need to be classified. Considering the difficulty of manual labeling, we manually add scene tags to only 20% of the total grid map cells. Then we employ the pre-trained DenseNet121 [25] for feature extraction and Support Vector Machines (SVM [26]) for classification. OpenCV [27] is also adopted to improve classification accuracy. After classification, the class labels are converted into heat values, and the grid map is converted into a heatmap affording concisely and efficiently distinguishing the importance of different cells. Fig. 3 (a) illustrates examples of the five classes and their corresponding heat-value in a heatmap, while Fig. 3 (b) presents our heatmap.

V. Our Algorithms

The Naive Greedy (Na-Greedy) algorithm is a processing method based on the vertex's Eff-weight. It suffers from a Weight Trap problem when employed on a single-UAV CPP scheme and a Weight Interference problem when utilized for multi-UAV CPP. Both problems force ending the coverage planning loop because the UAVs cannot find the next target vertex. The simplest solution is adding heuristic weights and developing the Heu-Greedy algorithm presented in Fig. 4 Algorithm 2. The corresponding idea is as follows, whenever a candidate for η_{nxt} is in CLOSE, a penalty of -10 is applied to its r to assist in covering a known vertex but without accumulating weight, preventing it from exiting the loop when a known vertex cannot be covered. Additionally, this

Authorized licensed use limited to: Hanoi University of Science & Technology. Downloaded on October 23,2026 at 08:26:45 UTC from IEEE Xplore. Restrictions apply.

strategy affords to distinguish each covered vertex. However, it reduces the search efficiency due to the repeated coverage, which is termed dynamic Weight Traps (repeatedly beating between the covered vertices). Other notations are listed below: current vertex η_c, candidate vertex η_{cd}, potential target vertex η_{SVeC}, acumulated Eff-weight R, distance of the nearest sufficient vertex min_{dis}.

Additionally, both Na-Greedy and Heu-Greedy algorithms suffer from the problems of Weight Sparse and Weight Interference, which will be introduced later. Although these problems do not affect coverage, they inject some invalid coverage. Hence, SVReC can be extended to a multi-UAV version by solving Weight Interference. We adopt the skip vertices coverage (SVeC) and the vertex weight resolution reinforcement (VwRR) methods to solve the above problems. Therefore, the overall algorithm is called SVReC. An overview of the above problems is illustrated in Fig. 5.

A. Weight Trap

Weight Trap occurs when all reachable eight-neighborhood vertices of the current vertex have been covered. The reachable eight-neighborhood vertices are the neighbor vertices that do not exceed the map range. The Weight Trap also takes place when we use heuristic weights as soft constraints. A UAV can choose the vertices that have already been covered as potential paths because as the r of vertices alternately descends, the vertices alternately become the next vertex in the path. As a result, the UAV ineffectively covers the area between these vertices. An example of a Weight Trap is depicted in Fig. 5 (a) above.

The Weight Trap problem can be solved utilizing SVeC, with the corresponding pseudo-code shown in Fig. 4 Algorithm 3. When the reachable eight-neighborhood vertices of the current vertex are all covered, the next UAV target η_{nxt} is specified as the nearest vertex to the current vertex, whose r exceeds a certain threshold r_{th}. Solving the Weight Trap problem is presented in Fig. 5 (a) below.

B. Weight Interference

Similar to Weight Trap, Weight Interference occurs when several UAVs collaborate to cover adjacent vertices. These vertices surround the current vertex of a UAV during multi-UAV coverage. In turn, during this case, the coverage stops or loops between the vertices that have been covered in (Fig. 5 (b) above).

SVeC can solve this problem as a collaborate strategy. Nevertheless, it is different when multiple UAVs are used, as all UAVs share the same CLOSE set during planning. The corresponding solution is illustrated in Fig. 5 (b) below.

C. Weight Sparse

When the critical areas of our heatmap are not continuous but are separated by some non-important areas, the Weight Sparse problem occurs. This is the case where the UAV is in an area where the average r of the uncovered reachable eight-neighborhood vertices of the current vertex is lower than a certain r_{th} and thus, the local solutions are not good enough. Weight Sparse reduces the search efficiency because a UAV has a poor probability of covering that area efficiently. For example, it is unreasonable to search in the water for people trapped by fire, and it is harmful to people who need to be rescued on land. An example is depicted in Fig. 5 (c) above.

Similarly, SVeC can also solve this type of problem. We designate UAV's next target η_{nxt}, which is the nearest vertex whose r is greater than a certain threshold r_{th} (Fig. 5 (c) below). In this way, the accumulated Eff-weight can be ensured as good as possible in the case of limited endurance because there is no need to consume UAV energy in areas with no relatively important vertices.

D. Weight Redundancy

When the above three problems are solved, Weight Redundancy may affect the quality of the local solutions because a UAV often faces several uncovered reachable vertices in the eight-neighborhood having the same r (Fig. 5 (d) above). These local solutions may be equivalent and redundant at the current processing step, but from the perspective of the overall MAEl-CPP task, they have apparent differences.

Thus, we employ the VwRR method to solve this problem, as illustrated in Fig. 5 (d) below with the corresponding pseudo-code presented in Fig. 4 Algorithm 4. The cells' average heat-value and the corresponding vertices' average Eff-weights in the green box are significantly higher than those in the blue and yellow boxes, so when η_{nxt} is specified as the green vertex **C** in the green box, we have a higher Eff-weight in the next few coverage steps because the potential efficiency is optimized.

VI. RESULTS & ANALYSIS

This section analyses and discusses the performance of different algorithms on solving the MAEl-CPP problem. Our experiments involve both multi-UAV and single-UAV setups. The algorithms challenged are the Zig-Zag, Na-Greedy, Heu-Greedy, and SVReC, while the evaluation metrics are the cover-rate σ, accumulated Eff-weight R, and the time required to accumulate a certain R.

A. Simulation Environment

To set up the simulation environment, we adopt the UE4 [28] and the Airsim [29]. Specifically, we load and edit the environment according to our task requirements in the UE4 engine and rasterize it into cells of $40m \times 40m$. Then we obtain the heatmap utilizing the method introduced in Section IV. For the CPP part, we calculate the directed subgraph γ in the graph structure through our algorithm and then convert it into waypoints. Waypoints are sent to the UAVs for position-based control, and then the UAV performs waypoints tracking and hovers over each cell that needs to be searched. We verify that our method can generate a heatmap based on the simulation environment, and our classifier's accuracy exceeds 91%, affording the UAVs to perform path planning successfully.

The four evaluated algorithms, i.e., Zig-Zag, Na-Greedy, Heu-Greedy, and SVReC, are applied to the heatmap of the

Authorized licensed use limited to: Hanoi University of Science & Technology. Downloaded on October 23,2026 at 08:26:45 UTC from IEEE Xplore. Restrictions apply.

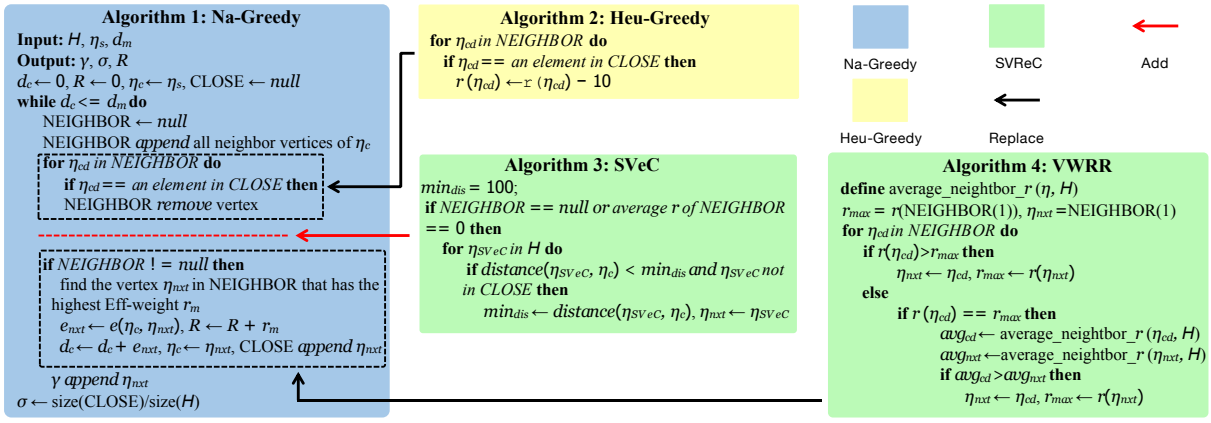


Fig. 4: Algorithm Overview

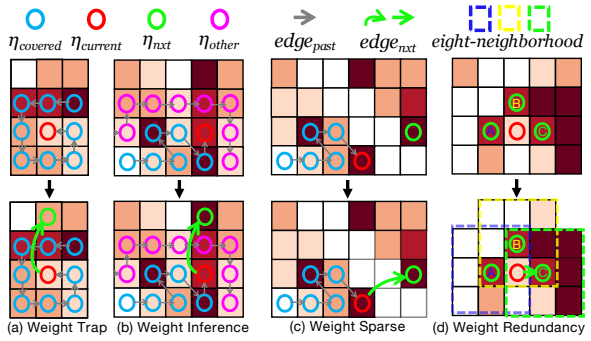


Fig. 5: Problems Overview

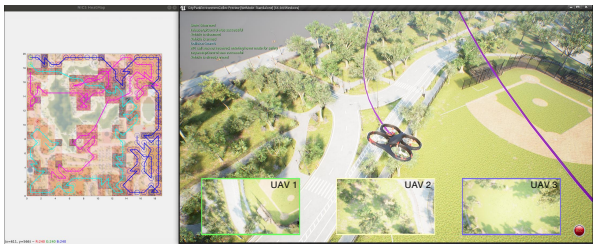


Fig. 6: Three UAV Simulation using SVReC in City Park

City Park [30] simulation environment. In the simulation environment, we simultaneously display the heatmap with the coverage paths and the UAV's downward-looking cameras, along with the flight status. This simulation environment manages a greater similarity to the real environment than random heatmaps. As illustrated in Fig. 6, we use three UAVs and set $d_m = 270$ for the simulation experiment. The SVReC algorithm covers most of the important areas in the simulation experiment.

B. Eff-weight Evaluation

For the Eff-weight Evaluation, the experimental settings per algorithms are shown in Tables I and II. In this trial we test the algorithms on two cases: $n = 1$ and $n = 6$. In the City Park heatmap, a single UAV is set to $d_m = 90$ and $d_m = 150$, while for the multi-UAVs scenario, $d_m = 180$ and $d_m = 300$. To further evaluate our algorithm's capabilities, we randomly generate several heatmaps. Although the latter heatmaps do not have a typical regional distribution like the simulation environment, they have a larger size, simulating extreme situations. For the trials, we utilize random heatmaps of sizes 50x50 and 100x100. If the grid scale is the same as the grid map of the simulation environment, the 100x100 heatmap mapped to the real environment is 16 square kilometers. In these maps, a single UAV is set to $d_m = 250$, $d_m = 550$, $d_m = 1100$ and $d_m = 2100$, while when multiple UAVs are involved, then $d_m = 450$, $d_m = 1050$, $d_m = 1800$ and $d_m = 4200$. For all maps, the endurance load of each UAV in the multi-UAV case is one-sixth of the d_m. A UAV starts from a randomly generated start vertex in each trial, and all algorithms use the same random start vertex in each experimental round. Once the coverage is over, we record the coverage rate and the cumulative Eff-weight.

Comparison: As highlighted in the columns under the label City Park in Tables I and II, when the map size is small, Heu-Greedy and SVReC perform better than Zig-Zag and Na-Greedy. SVReC adopts a logic that focuses more on potential trends and is more efficient in searching. So it performs better than its competitor algorithms. Although in some cases, the coverage rate of SVReC is slightly lower than the other algorithms, it still achieves a higher search efficiency demonstrating its efficiency in solving the Weight Redundancy and Weight Sparse problems. Therefore it can increase the probability of survivors being rescued and reduce the loss of life and property. Considering the random large-sized heatmaps, the columns under label 50×50 and 100×100 in Tables I and II highlight that the Na-Greedy algorithm performs poor due to the Weight Trap and Weight Interference, significantly reducing its coverage rate. Although the Heu-Greedy method successfully solves Weight Interference and avoids stopping the coverage, it suffers from the Weight Trap when r in the local optimal solution area alternately drops and invalid coverage is repeated between several vertices. Comparing both Tables I and II, we conclude that as the map size increases, SVReC demonstrates a more obvious advantage in R compared to its performance in the City Park map, which is significantly higher than the other

Authorized licensed use limited to: Hanoi University of Science & Technology. Downloaded on October 23,2026 at 08:26:45 UTC from IEEE Xplore. Restrictions apply.

TABLE I: Single UAV Experimental Results

UAV Number n=1		Map Size (cells)											
		City Park 19x20				50x50				100x100			
		d_m =90		d_m = 150		d_m=250		d_m=550		d_m=1100		d_m=2100	
		σ	ΣR_i	σ	ΣR_i	σ	ΣR_i	σ	ΣR_i	σ	ΣR_i	σ	ΣR_i
Algorithm	Zig-Zag	**23.7**%	282.1	**39.5**%	444.4	**10.0**%	426.5	**22.0**%	976.6	**11.0**%	2034.4	**21.0**%	3616.9
	Na-Greedy	21.1%	396.6	34.5%	596.8	7.1%	672.9	5.3%	476.4	3.0%	1200.0	4.0%	1578.5
	Heu-Greedy	21.1%	398.6	34.5%	660.3	8.2%	760.4	17.6%	1174.4	8.8%	2612.9	18.3%	4182.6
	SVReC	19.7%	**436.5**	31.8%	**698.9**	7.2%	**988.2**	15.2%	**1911.0**	7.4%	**3976.5**	14.2%	**7210.6**

TABLE II: Multi-UAV Experimental Results

UAV Number n=6		Map Size (cells)											
		City Park 19x20				50x50				100x100			
		d_m =180		d_m = 300		d_m=450		d_m=1050		d_m=1800		d_m=4200	
		σ	ΣR_i	σ	ΣR_i	σ	ΣR_i	σ	ΣR_i	σ	ΣR_i	σ	ΣR_i
Algorithm	Zig-Zag	**57.3**%	652.4	**67.7**%	920.7	**25.7**%	1091.0	**33.7**%	1348.4	**12.9**%	2216.6	16.8%	2780.9
	Na-Greedy	41.0%	665.3	67.2%	991.0	14.7%	934.6	27.0%	1691.9	4.6%	1130.6	6.6%	1752.3
	Heu-Greedy	40.1%	644.5	59.1%	892.0	14.1%	889.0	24.4%	1495.9	11.4%	2532.4	25.2%	5768.7
	SVReC	38.9%	**709.8**	61.7%	**1086.0**	13.9%	**1101.9**	29.0%	**2466.1**	12.8%	**4189.7**	**28.0**%	**9158.5**

TABLE III: Time Consuming for Algorithms to accumulate a certain R

n	1				6			
R	200	400	600	800	200	400	600	800
Zig-Zag	264.3	644.0	933.3	1270.0	240.6	574.2	844.4	1076.7
Na-Greedy	157.6	405.6	/	/	53.9	85.6	124.2	170.1
Heu-Greedy	157.6	405.6	603.1	810.9	58.4	85.1	126.4	178
SVREC	157.6	340.6	535.8	766.1	44.4	71.1	103.7	146.4

three algorithms. Its coverage rate is also higher than that of Na-Greedy and Heu-Greedy. This is because as the map size increases, SVReC avoids various problems that may lead to a stop of coverage or invalid coverage. When endurances are close, the average R per unit endurance of the multi-UAV SVReC is slightly lower than the single-UAV SVReC, but designing a single UAV set up with extremely long endurance is much more difficult than assigning these load endurances to multiple UAVs. Therefore, we propose the MAEl-CPP problem and the algorithms presented above.

C. Searching Time Evaluation

Regarding the evaluation of the searching time, we perform a speed racing experiment in the City Park environment and record per algorithm the average time in seconds required to accumulate a certain accumulated Eff-weight R. Our trials consider both the single-UAV and multi-UAV cases. The UAV's speed is set to $15m/s$, while for each cell center within the trajectory, we set a hovering time of 2 seconds. The corresponding results are presented in Table III.

Comparison: Comparing the last three columns with the third one in Table III, we conclude that Na-Greedy, Heu-Greedy, and SVReC require significantly less time than Zig-Zag for the same R, in both single-UAV and multi-UAV setups. Nevertheless, the single-UAV Na-Greedy cannot accumulate an Eff-weight more than 596 due to Weight Trap, Weight Sparse, and Weight Redundancy, as shown in the third and fourth columns of the Na-Greedy method. Under all conditions, SVReC performs better than the remaining algorithms, proving that the Eff-weight definition and SVReC's internal logic are reasonable. It is also worth noting that by comparing the columns under labels 1 and 6 for the MAEl-CPP problem, the multi-UAV case accumulates the same Eff-weight in a much shorter time than the single-UAV.

VII. Conclusion & Future Work

This paper proposes a new MAEl-CPP problem with explicit energy constraints and efficiency definitions, along with a heatmap that characterizes potential disaster risks and several MAEl-CPP algorithms. After several trials followed by analysis, the proposed SVREC algorithm is significantly better than the Na-Greedy, Heu-Greedy, and Zig-Zag algorithms when the endurance is limited. Additionally, SVREC is 4.2 times as efficient as the existing algorithm in terms of Eff-weight on large size maps.

However, our algorithm also has certain limitations. For example, for this type of NP-Hard problem, our algorithm's strategy cannot be guaranteed optimal. Secondly, our search and rescue algorithm is currently based on an offline *a priori* known environment and does not have online path planning capabilities if the UAV's downward-looking camera is fixed.

Future work shall involve deploying our algorithm on actual robots and terrain environments. Additionally, we will also focus on combining Micro-Macro control [31] on the UAV, aiming for more practical search and rescue actions.

Authorized licensed use limited to: Hanoi University of Science & Technology. Downloaded on October 23,2026 at 08:26:45 UTC from IEEE Xplore. Restrictions apply.

REFERENCES

[1] L. Ruan, J. Wang, J. Chen, Y. Xu, Y. Yang, H. Jiang, Y. Zhang, and Y. Xu, "Energy-efficient multi-uav coverage deployment in uav networks: A game-theoretic framework," *China Communications*, vol. 15, no. 10, pp. 194–209, 2018.

[2] H. Yang, R. Ruby, Q.-V. Pham, and K. Wu, "Aiding a disaster spot via multi-uav-based iot networks: Energy and mission completion time-aware trajectory optimization," *IEEE Internet of Things Journal*, pp. 1–1, 2021.

[3] E. T. Alotaibi, S. S. Alqefari, and A. Koubaa, "Lsar: Multi-uav collaboration for search and rescue missions," *IEEE Access*, vol. 7, pp. 55 817–55 832, 2019.

[4] M. Theile, H. Bayerlein, R. Nai, D. Gesbert, and M. Caccamo, "Uav coverage path planning under varying power constraints using deep reinforcement learning," 10 2020, pp. 1444–1449.

[5] O. Balotă, D. Iordan, G. Popescu, and D. Ilie, "Advanced uav lidar system for geospatial data collection," 11 2019.

[6] M.-h. Hwang, H.-R. Cha, and S. Y. Jung, "Practical endurance estimation for minimizing energy consumption of multirotor unmanned aerial vehicles," *Energies*, vol. 11, no. 9, 2018. [Online]. Available: https://www.mdpi.com/1996-1073/11/9/2221

[7] Z. Cao, H. Gao, K. Mangalam, Q.-Z. Cai, M. Vo, and J. Malik, "Long-term human motion prediction with scene context," in *Computer Vision – ECCV 2020*, A. Vedaldi, H. Bischof, T. Brox, and J.-M. Frahm, Eds. Cham: Springer International Publishing, 2020, pp. 387–404.

[8] N.-T. Nguyen and B.-H. Liu, "The mobile sensor deployment problem and the target coverage problem in mobile wireless sensor networks are np-hard," *IEEE Systems Journal*, vol. 13, no. 2, pp. 1312–1315, 2019.

[9] A. Pintado and M. Santos, "A first approach to path planning coverage with multi-uavs," in *15th International Conference on Soft Computing Models in Industrial and Environmental Applications (SOCO 2020)*, Á. Herrero, C. Cambra, D. Urda, J. Sedano, H. Quintián, and E. Corchado, Eds. Cham: Springer International Publishing, 2021, pp. 667–677.

[10] E. Gonzalez, O. Alvarez, Y. Diaz, C. Parra, and C. Bustacara, "Bsa: A complete coverage algorithm," in *Proceedings of the 2005 IEEE International Conference on Robotics and Automation*, 2005, pp. 2040–2044.

[11] Y. Gabriely and E. Rimon, "Spiral-stc: An on-line coverage algorithm of grid environments by a mobile robot," in *Proceedings 2002 IEEE International Conference on Robotics and Automation (Cat. No. 02CH37292)*, vol. 1. IEEE, 2002, pp. 954–960.

[12] B. Kalyanasundaram and K. R. Pruhs, "Constructing competitive tours from local information," *Theoretical Computer Science*, vol. 130, no. 1, pp. 125–138, 1994.

[13] J. Tang, C. Sun, and X. Zhang, "Mstc*: multi-robot coverage path planning under physical constraints," *arXiv preprint arXiv:2108.04632*, 2021.

[14] Y. Zhou, L. Kong, S. Sosnowski, Q. Liu, and S. Hirche, "Distributed event- and self-triggered coverage control with speed constrained unicycle robots," *CoRR*, vol. abs/2107.14580, 2021. [Online]. Available: https://arxiv.org/abs/2107.14580

[15] M. Torres, D. A. Pelta, J. L. Verdegay, and J. C. Torres, "Coverage path planning with unmanned aerial vehicles for 3d terrain reconstruction," *Expert Systems with Applications*, vol. 55, pp. 441–451, 2016. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0957417416300306

[16] D. Panagou, D. M. Stipanović, and P. G. Voulgaris, "Distributed dynamic coverage and avoidance control under anisotropic sensing," *IEEE Transactions on Control of Network Systems*, vol. 4, no. 4, pp. 850–862, 2016.

[17] Q. An and Y. Shen, "Distributed coverage control for mobile camera sensor networks with anisotropic perception," *IEEE Sensors Journal*, 2021.

[18] L. Lin and M. A. Goodrich, "Uav intelligent path planning for wilderness search and rescue," in *2009 IEEE/RSJ International Conference on Intelligent Robots and Systems*. IEEE, 2009, pp. 709–714.

[19] C. Di Franco and G. Buttazzo, "Energy-aware coverage path planning of uavs," in *2015 IEEE International Conference on Autonomous Robot Systems and Competitions*, 2015, pp. 111–117.

[20] T. Dang, F. Mascarich, S. Khattak, C. Papachristos, and K. Alexis, "Graph-based path planning for autonomous robotic exploration in subterranean environments," in *2019 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2019, pp. 3105–3112.

[21] P. Wojtaszczyk, "Greedy algorithm for general biorthogonal systems," *Journal of Approximation Theory*, vol. 107, no. 2, pp. 293–314, 2000. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0021904500935123

[22] J. Zhou, X. Zhao, X. Zhang, D. Zhao, and H. Li, "Task allocation for multi-agent systems based on distributed many-objective evolutionary algorithm and greedy algorithm," *IEEE Access*, vol. 8, pp. 19 306–19 318, 2020.

[23] P. G. Luan and N. T. Thinh, "C 2 piecewise cubic bezier curve based smoothing path for mobile robot," *International Journal of Mechanical Engineering and Robotics Research*, vol. 10, no. 9, 2021.

[24] C. O'Meadhra, W. Tabib, and N. Michael, "Variable resolution occupancy mapping using gaussian mixture models," *IEEE Robotics and Automation Letters*, vol. 4, no. 2, pp. 2015–2022, 2018.

[25] G. Huang, Z. Liu, L. van der Maaten, and K. Q. Weinberger, "Densely connected convolutional networks," 2018.

[26] X. Li, L. Wang, and E. Sung, "Multilabel svm active learning for image classification," in *2004 International Conference on Image Processing, 2004. ICIP'04.*, vol. 4. IEEE, 2004, pp. 2207–2210.

[27] "Home," Jun 2021. [Online]. Available: https://opencv.org/

[28] "The most powerful real-time 3d creation platform." [Online]. Available: https://www.unrealengine.com/en-US/

[29] Microsoft, "microsoft/airsim: Open source simulator for autonomous vehicles built on unreal engine / unity, from microsoft ai & research." [Online]. Available: https://github.com/Microsoft/AirSim

[30] "City park environment collection in environments - ue marketplace." [Online]. Available: https://www.unrealengine.com/marketplace/en-US/product/city-park-environment-collection

[31] A. Sharon, N. Hogan, and D. E. Hardt, "The macro/micro manipulator: An improved architecture for robot control," *Robotics and Computer-Integrated Manufacturing*, vol. 10, no. 3, pp. 209–222, 1993. [Online]. Available: https://www.sciencedirect.com/science/article/pii/073658459390056P

Authorized licensed use limited to: Hanoi University of Science & Technology. Downloaded on October 23,2026 at 08:26:45 UTC from IEEE Xplore. Restrictions apply.